

TEST PLAN

Based on ISO/IEC/IEEE 29119-2 Section 7.2

Application Under Test

dr.com.tr – Books / New Releases

https://www.dr.com.tr/kategori_/kitap/en-yeniler/10001/3

Prepared by

Mehmet Dünder

Batu Pala

Ilia Sergeev

Demir Ege Şimşek

Submission Deadline: May 4, 2025

1. Context of Testing

1.1 Project / Test Levels / Test Types

This test plan covers the SE 2226 term project. Two applications are tested. **App 1** is the dr.com.tr e-commerce web application (Books → New Releases section), tested at **System Test** level using black-box techniques automated with Selenium WebDriver. Mehmet and Batu automate tests in **Java + Maven** (selenium-java:4.40.0, as per the course Week 7–8 toolset). Ilia automates equivalent tests in **Python + Selenium WebDriver** (personal tool preference; identical test cases, different language). **App 2** is Library-DB-Express (Node.js/Express REST API), tested at **Unit Test** level with **JUnit 6**, IntelliJ built-in coverage (Week 10), and **PIT mutation testing** in Eclipse (Week 13).

1.2 Test Items

#	Application	Description	Reference / Version
1	dr.com.tr	Turkish e-commerce platform. Focus: Books → New Releases. Modules under test: Search, Filters, Product Detail, Shopping Cart, Login / Register, Checkout.	Live production site – April 2025 https://www.dr.com.tr/kategori_kitap/en-yeniler/10001/3
2	Library-DB-Express	Node.js / Express REST API for a library system. Provides CRUD endpoints for books, members, and loans. Source from GitHub (jlayher/Library-DB-Express).	GitHub – latest commit at test start date

1.3 Test Scope

App 1 – dr.com.tr — features to be tested:

- Book search by keyword – valid query, empty query, only spaces, special characters
- Search filters – price range slider (boundary values), author, publisher, category
- Product listing page – pagination (first / middle / last page) and sort order
- Product detail page – price and stock status displayed correctly; Add to Cart button
- Shopping cart – add item, update quantity (BVA: 0, 1, 50, 99, 100), remove item
- User login – valid credentials, wrong password, unregistered e-mail, empty fields
- Checkout flow – address form field validation (required fields, invalid formats)

Excluded from App 1 testing (rationale):

- Server-side code and database of dr.com.tr – black-box only; no access
- Real payment transactions – checkout UI form is tested, no real card charges
- Mobile app (iOS / Android) – Chrome browser via Selenium only
- Admin / back-office panel – not user-facing, no credentials

App 2 – Library-DB-Express — features to be tested:

- REST endpoints: GET, POST, PUT, DELETE for /books, /members, /loans
- HTTP status codes and response body correctness for all endpoints
- Input validation: missing required fields, wrong data types, out-of-range values
- Statement and branch coverage – measured with IntelliJ built-in coverage runner

- Mutation score – measured with PIT plugin in Eclipse IDE

1.4 Test Basis

- SE 2226 lecture materials and weekly lab tools (Selenium Week 6–8, JUnit 6 Week 2–5, Coverage Week 10, PIT Week 13)
- Observed behaviour of the live dr.com.tr website
- Library-DB-Express source code, README, and API endpoint documentation
- ISO/IEC/IEEE 29119 standard for software testing

2. Assumptions and Constraints

Category	Description
Assumption 1	dr.com.tr remains functionally stable during the testing period (no major redesign or URL changes).
Assumption 2	Mehmet, Batu and Demir have Java 17+, Maven, IntelliJ Community, and Eclipse installed.
Assumption 3	Ilia has Python 3.10+ with the selenium package installed as a personal alternative to Java Selenium.
Assumption 4	Library-DB-Express runs locally on localhost:3000 with Node.js v18+.
Constraint 1	Budget: \$0. Only free / open-source tools are used (Java, Maven, Python, Selenium, JUnit 6, PIT, IntelliJ Community, Eclipse).
Constraint 2	Timeline: all testing must be complete before the May 4, 2025 presentation.
Constraint 3	Team size: 4 students. Work is divided equally (~5 working days per person).
Constraint 4	No backend source access for dr.com.tr – all App 1 tests are black-box.
Constraint 5	Real credit card data cannot be used; only the checkout UI form is verified.

3. Test Strategy

3.1 General

App 1 uses a **black-box, automation-first** approach. Every test case is implemented as a runnable Selenium WebDriver script – not executed manually. Mehmet and Batu write tests in **Java (Maven, selenium-java:4.40.0)**, following the course Week 7–8 setup. Ilia writes the same test cases in **Python (selenium package)** as a personal language preference. All four members share the same test case design (ECP tables, BVA tables, decision table, use-case scenarios) agreed upon before implementation. App 2 uses a **white-box** approach, primarily executed by Demir with JUnit tests implemented by Ilia: JUnit 6 unit tests, IntelliJ built-in coverage, and PIT mutation testing in Eclipse.

3.2 Test Levels

Application	Test Level	Justification
dr.com.tr (App 1)	System Testing	The complete web application is driven end-to-end through a real Chrome browser via Selenium WebDriver.
Library-DB-Express (App 2)	Unit Testing	Individual API route handler functions are tested in isolation via JUnit 6 running against the local Node.js server.

3.3 Test Types

Test Type	App	Tool	Description
Functional Testing (automated GUI)	App 1	Java Selenium (Mehmet, Batu) Python Selenium (Ilia)	Each module (search, cart, login, checkout) is verified against expected behaviour through a real Chrome browser.
Equivalence Class Partitioning	App 1	Selenium + parametrized test cases	Valid / invalid / edge input partitions: search queries, login fields, cart quantity. Each partition = one test case.
Boundary Value Analysis	App 1	Selenium + parametrized test cases	Cart quantity: 0 (invalid), 1 (min), 50 (mid), 99 (max), 100 (invalid). Price filter slider: lower and upper boundary values.
Use-Case / State Transition Testing	App 1	Selenium multi-step script	Full user flow: Guest → Login → Search book → Add to Cart → Checkout form. Each state transition is asserted.
Decision Table Testing	App 1	Selenium parametrized	Login: 4 combinations of valid/invalid e-mail x valid/invalid password. Each combination is an automated test case.
Unit Testing	App 2	JUnit 6 (Java, IntelliJ)	Test class for Library-DB-Express: one test method per endpoint and per edge case. HTTP calls via <code>java.net.http.HttpClient</code> .
Coverage Analysis	App 2	IntelliJ built-in coverage runner	Statement and branch coverage measured by running the JUnit suite with IntelliJ coverage. Target: $\geq 70\%$ statement, $\geq 60\%$ branch.
Mutation Testing	App 2	PIT plugin (Eclipse IDE)	PIT introduces faults into the source code. Surviving mutants are documented and analysed. Target score: $\geq 60\%$.

3.4 Test Deliverables

- This Test Plan (PDF)
- Test Case Specification: ECP tables, BVA tables, decision table, use-case scenario (shared document)
- Java Selenium test classes (Mehmet & Batu): `SearchTest.java`, `LoginTest.java`, `CartTest.java`, `CheckoutTest.java`
- Python Selenium test scripts (Ilia): `test_search.py`, `test_login.py`, `test_cart.py`, `test_checkout.py`
- JUnit 6 test class for Library-DB-Express (Ilia): `LibraryApiTest.java`

- IntelliJ coverage HTML report (statement and branch coverage)
- PIT mutation HTML report (mutation score and surviving mutants)
- Test Completion Report
- Presentation slides

3.5 Test Design Techniques

Technique	Applied To	Concrete Example
Equivalence Class Partitioning	Search, Login, Cart Quantity	Search input partitions: (1) valid book title, (2) empty string, (3) only spaces, (4) special characters. Login partitions: (1) valid email, (2) unregistered email, (3) empty field.
Boundary Value Analysis	Cart quantity, Price filter	Cart quantity: test values 0, 1, 50, 99, 100. Values 0 and 100 are expected to be rejected; 1-99 are valid. Price filter slider: min=0 TL, max boundary of the range.
Use-Case / State Transition	Full purchase flow	States: Guest → Logged In → Item in Cart → Checkout Page → Order Confirmed. Each transition is triggered and asserted in a single multi-step script.
Decision Table	Login module	4 rules: (valid email, valid pwd) → success; (valid email, wrong pwd) → error; (invalid email format, any) → error; (empty, empty) → validation message.
Statement & Branch Coverage	Library-DB-Express (App 2)	JUnit 6 suite is run with IntelliJ coverage runner. Uncovered lines are identified and additional test cases are added until $\geq 70\%$ statement coverage is reached.
Mutation Testing (PIT)	Library-DB-Express (App 2)	PIT (Eclipse plugin) mutates operators and conditions in the Java source. Killed vs. surviving mutants are reported. Target: $\geq 60\%$ mutation score.

3.6 Entry and Exit Criteria

Entry criteria – test execution may begin when:

- dr.com.tr is accessible in Chrome and the basic user flow is confirmed working
- Java Selenium project compiles and a smoke WebDriver test opens the browser (Mehmet & Batu)
- Python Selenium smoke test opens Chrome and loads dr.com.tr without error (Ilia)
- Library-DB-Express runs on localhost:3000 and responds to a manual GET /books request
- JUnit 6 project compiles and one sample test passes in IntelliJ
- PIT plugin is installed in Eclipse and can scan the project without errors
- All test cases (ECP/BVA tables, decision table, use-case) are written and reviewed by the team

Exit criteria – testing may stop when:

- All Selenium test cases (Java and Python) have been executed and results recorded
- JUnit 6 suite achieves $\geq 70\%$ statement coverage (IntelliJ report generated)
- PIT mutation score is $\geq 60\%$ and the HTML report is saved as a deliverable
- All discovered defects are documented in the incident log

3.7 Test Completion Criteria

- All ECP, BVA, decision table, and use-case test cases have been executed and actual results are recorded
- State transition (multi-step) Selenium script runs the full Guest → Checkout flow without crashing
- JUnit 6 suite: $\geq 70\%$ statement coverage and $\geq 60\%$ branch coverage (IntelliJ report)
- PIT mutation score $\geq 60\%$; surviving mutants are listed and commented in the report
- Test Completion Report is finalized and ready for submission before May 4, 2025

3.8 Metrics to be Collected

Metric	Description	Target
Test cases designed	Total number of test cases across all techniques (ECP, BVA, use-case, decision table)	≥ 30 for App 1
Test cases executed	Number of designed test cases actually run	100%
Pass / Fail count	How many test cases pass; failures are treated as defect candidates	All executed; failures logged
Defects found	Anomalies discovered during test execution (unexpected UI behaviour, crashes, wrong output)	All logged
Statement coverage	% of Library-DB-Express source lines covered by the JUnit 6 suite	$\geq 70\%$
Branch coverage	% of conditional branches covered by the JUnit 6 suite	$\geq 60\%$
Mutation score	% of PIT mutants killed by the JUnit 6 suite	$\geq 60\%$

3.9 Test Data Requirements

Data Type	Details	Responsible
Valid dr.com.tr account	A real registered account (email + password) stored in a local config file outside version control. Used for login and checkout tests.	Mehmet Dundar
Search query list	Parametrized list: a known book title (e.g. a current bestseller), empty string, string of spaces, string with special characters. Used across all three Selenium implementations.	Batu Pala
Cart quantity values	List of boundary values: [0, 1, 50, 99, 100]. Used in the cart BVA test cases.	Batu Pala
Login credential matrix	4 tuples covering the decision table: (valid email, valid pwd), (valid email, wrong pwd), (bad email format, any pwd), (empty, empty).	Mehmet Dundar
Library-DB seed data	JSON fixtures (sample books, members, loan records) loaded before each JUnit test class via @BeforeAll to populate the local database.	Ilia Sergeev

3.10 Test Environment Requirements

Component	Version / Details	Used by
Google Chrome	Latest stable version	All members (Selenium WebDriver)
ChromeDriver	Must match Chrome version. Use WebDriverManager (Java) or webdriver-manager (Python) for auto-management.	All members
Java JDK	Oracle JDK 8 or JDK 17+ (as per course Week 1 setup)	Mehmet, Batu (Selenium Java); Ilia (JUnit 6)
Maven	org.seleniumhq.selenium:selenium-java:4.40.0 (Week 7-8 course setup)	Mehmet, Batu
IntelliJ Community IDE	For writing and running JUnit 6 tests with built-in coverage runner (Week 10)	Demir
JUnit 6 (Jupiter)	org.junit.jupiter:junit-jupiter:6.0.2 + org.junit.platform:junit-platform-launcher:6.0.2 (Week 2-5 course setup)	Ilia, Demir
Eclipse IDE for Java Developers	With PIT plugin installed (Week 13 course setup)	Demir
Python 3.10+	With selenium package installed via pip	Ilia only
Node.js	v18 LTS – to run Library-DB-Express on localhost:3000	Ilia

3.11 Testing Activities and Estimates

#	Activity	Est.	Owner
1	Set up all environments: Java+Maven+Selenium, Python+Selenium, Node.js+Library-DB-Express, JUnit 6 in IntelliJ, PIT in Eclipse	1 day	All
2	Design test cases together: ECP table, BVA table, login decision table, use-case / state transition scenario	2 days	All
3	Implement Java Selenium test classes: SearchTest.java, LoginTest.java (ECP + decision table)	3 days	Mehmet
4	Implement Java Selenium test classes: CartTest.java, CheckoutTest.java (BVA + use-case / state transition multi-step script)	3 days	Batu
5	Implement Python Selenium test scripts covering the same test cases: test_search.py, test_login.py, test_cart.py, test_checkout.py	3 days	Ilia
6	Write JUnit 6 test class (LibraryApiTest.java) for all Library-DB-Express endpoints; run and fix until passing	2 days	Ilia
7	Run IntelliJ coverage; identify uncovered lines; add tests until $\geq 70\%$ statement coverage	1 day	Demir
8	Configure and run PIT in Eclipse; document surviving mutants; aim for $\geq 60\%$ mutation score	2 days	Demir

#	Activity	Est.	Owner
9	Execute full Selenium suites (Java + Python); record all results; log defects	1 day	All
10	Write Test Completion Report; compile all HTML reports as attachments	2 days	All
11	Prepare and rehearse presentation slides	1 day	All

Total estimate: ~21 person-days across 4 members (~5 days each). Budget: \$0 (all tools are free / open-source).

4. Staffing

4.1 General

Testing is carried out by 4 students. Each member owns one primary technical area and provides secondary support across all other activities. Test case designs (ECP, BVA, decision table, use-case) are agreed by the whole team before individual implementation begins.

4.2 Roles and Responsibilities

Member	Primary Role	Primary Responsibilities	Secondary Support
Mehmet Dundar	Selenium Lead (Java) – Search & Login	Design ECP table and login decision table. Implement SearchTest.java and LoginTest.java in Java + Maven. Execute and record results.	Review Batu's test cases. Help compile the Test Completion Report.
Batu Pala	Selenium Lead (Java) – Cart & Checkout	Design BVA table and use-case / state transition scenario. Implement CartTest.java and CheckoutTest.java in Java + Maven. Run the full multi-step end-to-end script.	Review Mehmet's test cases. Help prepare presentation slides.
Ilia Sergeev	Python Selenium + Unit Test Lead	Implement the same App 1 test cases in Python + Selenium (test_search.py, test_login.py, test_cart.py, test_checkout.py). Set up Library-DB-Express locally. Write JUnit 6 suite.	Assist App 1 test execution review. Compile all HTML reports for submission.
Demir Ege Şimşek	Coverage & Mutation Testing Lead	Execute code coverage analysis using IntelliJ. Run PIT mutation testing in Eclipse.	Assist Ilia in improving unit test coverage by identifying gaps and proposing additional test cases.

4.3 Schedule

Phase	Activities	Dates	Milestone
1 – Setup & Planning	Configure environments (Java, Python, Node.js). Finalize and submit Test Plan.	Apr 14–18	Test Plan submitted
2 – Setup & Planning	Update Test Plan to redistribute responsibilities.	Apr 18–21	Test Plan updated.
3 – Test Design	Whole team: write ECP table, BVA table, decision table, use-case scenario document.	Apr 21–23	Test Case Spec ready
Phase	Activities	Dates	Milestone
4 – Selenium Implementation	Mehmet: SearchTest.java + LoginTest.java. Batu: CartTest.java + CheckoutTest.java. Ilia: test_search.py + test_login.py + test_cart.py + test_checkout.py.	Apr 24 – May 1	All Selenium suites runnable
5 – Unit Tests & Coverage	Ilia: LibraryApiTest.java. Demir: IntelliJ coverage report → PIT mutation report in Eclipse.	Apr 24 – May 1	Coverage & mutation reports
6 – Execution & Reporting	Run full test suites; log defects; write Test Completion Report; attach all HTML reports.	May 2–3	Completion Report done
7 – Presentation	Rehearse and deliver project presentation on May 8.	May 8, 2025	Presentation delivered